

HW4

STAT 651

RJ Cass

2026-02-13

1: BDA 3.1

a)

$$(y_i, \dots, y_J) \sim \text{Multinom}(\theta_1, \dots, \theta_J)$$

$$\theta = (\theta_1, \dots, \theta_J), \pi(\theta) = \text{Dir}_{\sim}(a)$$

$$\alpha = \frac{\theta_1}{\theta_1 + \theta_2}$$

Because the Dirichlet is conjugate to the Multinomial, we know that $p(\theta|y) = f(y|\theta)\pi(\theta) = \text{Dir}(a')$ where $a' = a + y$

$$\Rightarrow \pi(\theta|y) = \text{Dir}_{\sim}(y + a)$$

Dirichlet's can be decomposed into other Dirichlet's, where basically: $\theta_1 = \text{Dir}(a_1, 1 - a_1)$, thus we can decompose $\pi(\theta|y) = \text{Dir}(a_1 + y_1, a_2 + y_2, 1 - ((a_1 + y_1) + (a_2 + y_2)))$. The last term is just all the other Dirichlet components besides θ_1, θ_2 . We'll just call those 'other'. $\Rightarrow \propto \theta_1^{a_1+y_1-1} \theta_2^{a_2+y_2-1} (1 - \theta_1 - \theta_2)^{a_{\text{other}}+y_{\text{other}}-1}$

From here, I talked with classmates to see I need to do the following transform: $\alpha = \frac{\theta_1}{\theta_1 + \theta_2}, \beta = \theta_1 + \theta_2$

$$\Rightarrow \theta_2 = \beta - \theta_1 \Rightarrow \alpha = \frac{\theta_1}{\theta_1 + \beta - \theta_1} \Rightarrow \theta_1 = \alpha\beta \Rightarrow \theta_2 = \beta - \alpha\beta = \beta(1 - \alpha)$$

Plugging those into the above function:

$$\begin{aligned} p(\alpha, \beta|y) &\propto (\alpha\beta)^{a_1+y_1-1} (\beta(1-\alpha))^{a_2+y_2-1} (1 - \alpha\beta - \beta(1-\alpha))^{a_{\text{other}}+y_{\text{other}}-1} \\ &= \alpha^{a_1+y_1-1} \beta^{a_1+y_1-1} (1-\alpha)^{a_2+y_2-1} (\beta)^{a_2+y_2-1} (1-\beta)^{a_{\text{other}}+y_{\text{other}}-1} \\ &= \alpha^{a_1+y_1-1} (1-\alpha)^{a_2+y_2-1} (\beta)^{a_2+y_2+a_1+y_1-2} (1-\beta)^{a_{\text{other}}+y_{\text{other}}-1} \\ &\propto \text{Beta}(\alpha|a_1 + y_1, a_2 + y_2) \text{Beta}(\beta|stuff) \end{aligned}$$

Thus, $p(\alpha|\theta_1, \theta_2) = \text{Beta}(a_1 + y_1, a_2 + y_2)$

b)

I talked with classmates to realize I am supposed to use a $Beta(a_1, a_2)$ prior for α

$$f(y_1|n = y_1 + y_2, p = \alpha)\pi(\alpha) = \alpha^{y_1}(1 - \alpha)^{y_1+y_2-y_1}Beta(a_1, a_2) \propto \alpha^{y_1}(1 - \alpha)^{y_2}\alpha^{a_1}(1 - \alpha)^{a_2} = \alpha^{y_1+a_1}(1 - \alpha)^{y_2+a_2} \propto Beta(a_1 + y_1, a_2 + y_2)$$

2 BDA3 3.2

Because the Dirichlet is the conjugate for the Multinomial, the Posteriors will be Dirichlet, with probabilities equal to the ratio of each category:

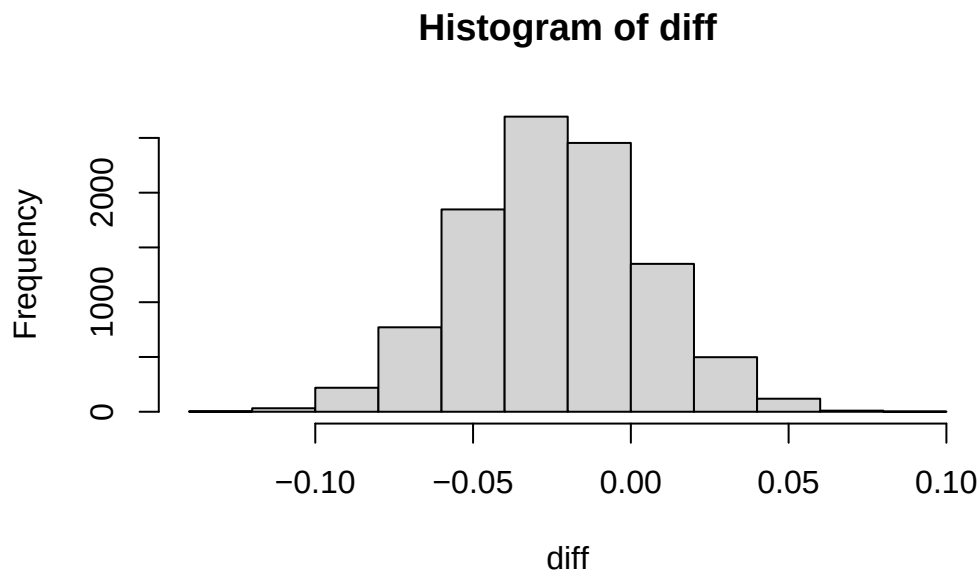
$$\pi(\theta_1, \theta_2, \theta_3|y) = Dir(294, 307, 38), \pi(\eta_1, \eta_2, \eta_3|y) = Dir(288, 332, 19)$$

In this case we only care about the proportion of those for Bush out of those who picked Bush/Dukakis:

$\alpha_1 = \frac{\theta_1}{\theta_1 + \theta_2}, \alpha_2 = \frac{\eta_1}{\eta_1 + \eta_2}$. This is the same setup as the previous question, we'll use a $Beta(1, 1)$ prior, meaning the posterior for each is a Beta:

$$\pi(\alpha_1|y) = Beta(295, 308), \pi(\alpha_2|y) = Beta(289, 333)$$

```
n_samp <- 10e3
alpha1 <- rbeta(n_samp, 295, 308)
alpha2 <- rbeta(n_samp, 289, 333)
diff <- alpha2 - alpha1
hist(diff)
```



```
change <- mean(diff > 0)
```

The histogram of differences is above. The posterior probability that there was a shift towards Bush is 0.1979.

3 BDA3 3.11

a)

```
suppressPackageStartupMessages(library(MASS))
suppressPackageStartupMessages(library("rstan"))
suppressPackageStartupMessages(library(ggplot2))

dt <- data.frame(dose = c(-.86, -.3, -.05, .73),
                 n = c(5, 5, 5, 5),
                 y = c(0, 1, 3, 5))

# Prior values
m_a <- 0
v_a <- 2^2
m_b <- 10
v_b <- 10^2
corr <- .5

mu <- c(m_a, m_b)          # Mean vector (X1, X2)
Sigma <- matrix(c(v_a, corr, corr, v_b), 2, 2) # Covariance matrix

n_sim <- 1000

# samp <- mvrnorm(n = n_sim, mu = mu, Sigma = sigma)
# alphas <- samp[, 1]
# betas <- samp[, 2]

xx <- seq(-1, 1, length = 200)

#### STAN MODEL
data_stan <- list(N = length(dt$y), y = dt$y, x = dt$dose, n_vec = dt$n, mu = mu, L_Sigma = ch

params <- c("alpha", "beta")

n_cores <- parallel::detectCores() # Count how many cores are available
options(mc.cores = n_cores)

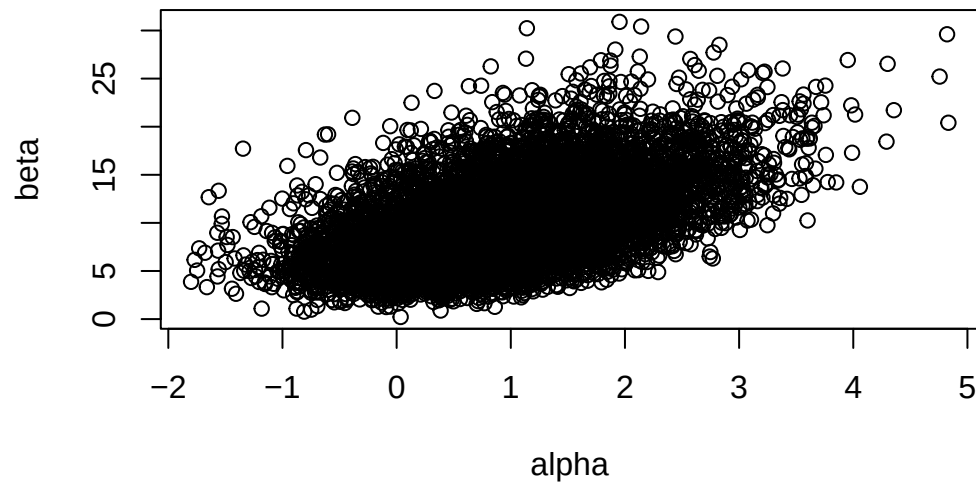
fit_stan <- stan(model_code = readLines("q3.stan"),
                 data = data_stan, pars = params,
                 iter = 5000, warmup = 500, thin = 5, chains = n_cores)

sim_stan <- extract(fit_stan) # collect samples from all chains

alpha <- sim_stan$alpha
beta <- sim_stan$beta
```

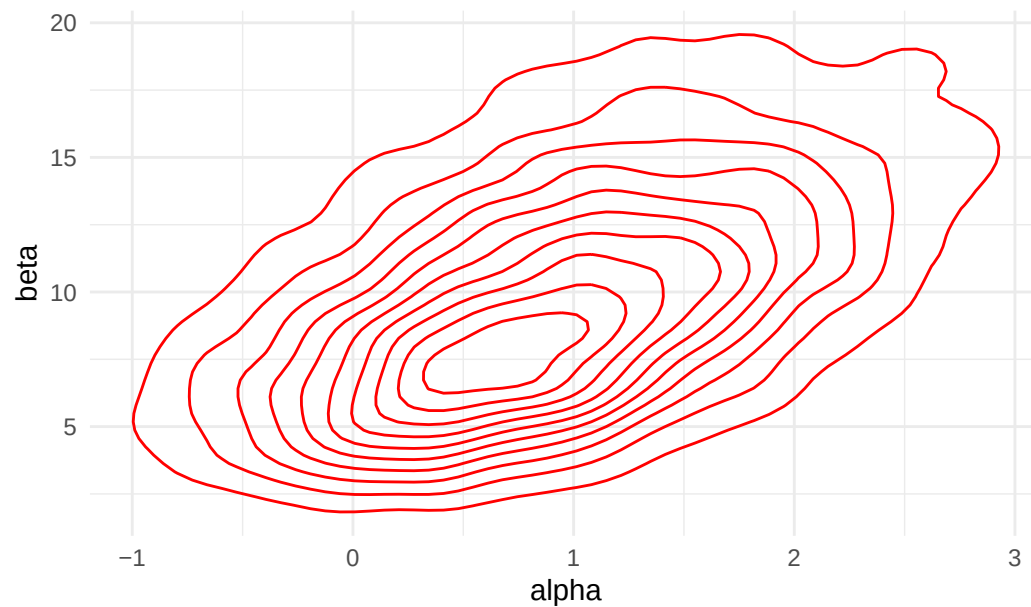
```
plot(sim_stan$alpha, sim_stan$beta, xlab = 'alpha', ylab = 'beta', main = 'Scatterplot')
```

Scatterplot



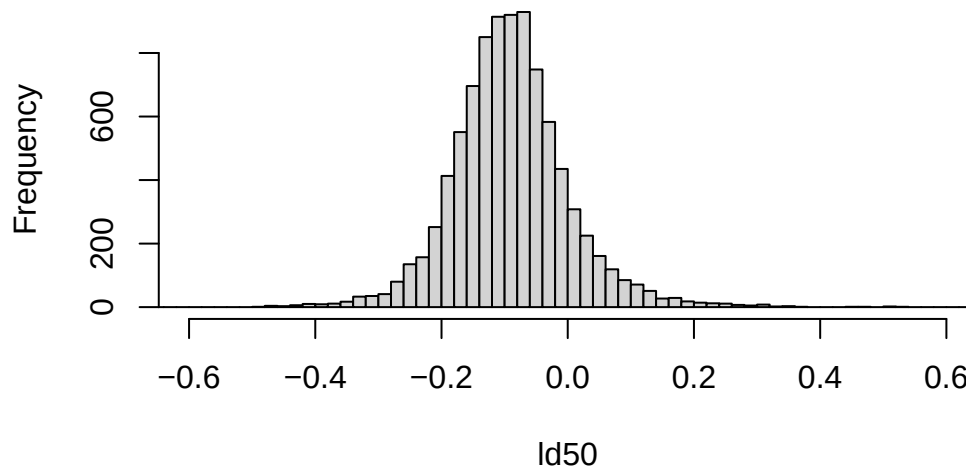
```
ggplot(data = NULL, aes(x = sim_stan$alpha, y = sim_stan$beta)) +  
  # Add 2D density contour lines  
  geom_density_2d(color = "red") +  
  theme_minimal() +  
  labs(title = "Contour Plot",  
        x = "alpha",  
        y = "beta")
```

Contour Plot



```
ld50 <- -sim_stan$alpha/sim_stan$beta
hist(ld50, breaks = 100, xlim = c(-0.6, 0.6))
```

Histogram of ld50



STAN code:

```
data {
  int<lower=0> N;
  array[N] int y;
  vector[N] x; //dose
  array[N] int n_vec;

  vector[2] mu;
  matrix[2, 2] L_Sigma; // Cholesky factor of covariance
}

parameters {
  vector[2] theta;
}

transformed parameters {
  // Put into 2 different variables
  real alpha;
  real beta;
  alpha = theta[1];
  beta = theta[2];
}

model {
  vector[N] theta_vec = inv_logit(alpha + beta * x);

  y ~ binomial(n_vec, theta_vec);
}
```

```

    theta ~ multi_normal_cholesky(mu, L_Sigma);
}

```

See plots above, that they are similar to the plots in the book

b)

```

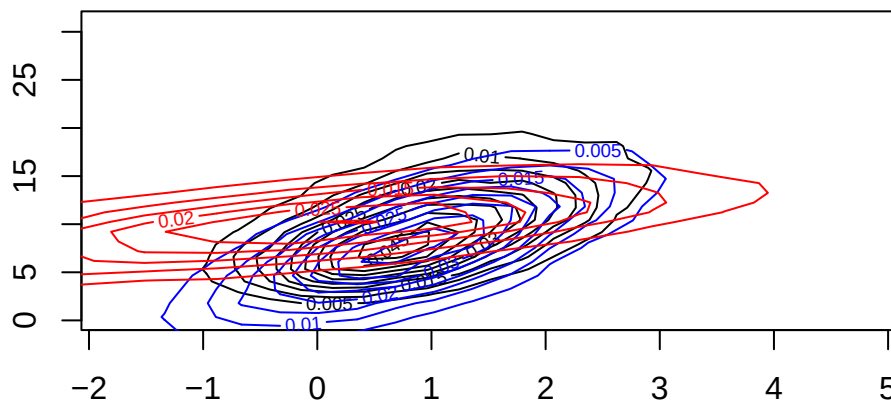
mod <- glm(dt$y/dt$n ~ dt$dose, family = binomial, weights = dt$n)
mu_hat <- coef(mod)
sigma_hat <- vcov(mod)

lik_z <- mvrnorm(10e3, mu = mu_hat, Sigma = sigma_hat)

prior_z <- mvrnorm(10e3, mu = c(0, 10), Sigma = matrix(c(4, 10), c(10, 100), nrow = 2, ncol = 2))

kde_lik <- kde2d(lik_z[, 1], lik_z[, 2])
kde_prior <- kde2d(prior_z[, 1], prior_z[, 2])
kde_post <- kde2d(sim_stan$alpha, sim_stan$beta)
contour(kde_post$x, kde_post$y, kde_post$z)
contour(kde_lik$x, kde_lik$y, kde_lik$z, add = TRUE, col = 'blue')
contour(kde_prior$x, kde_prior$y, kde_prior$z, add = TRUE, col = 'red')

```



c)

The resulting histogram appears to be a bit more shifted towards 0, but not significantly different. They are surprisingly similar to the in-book plots. Likely means with enough samples the effect of prior disappears.

For the extra HW portions:

For parts a) and b), it's not clear what we're supposed to do with the draws, so I made a histogram for each

a)

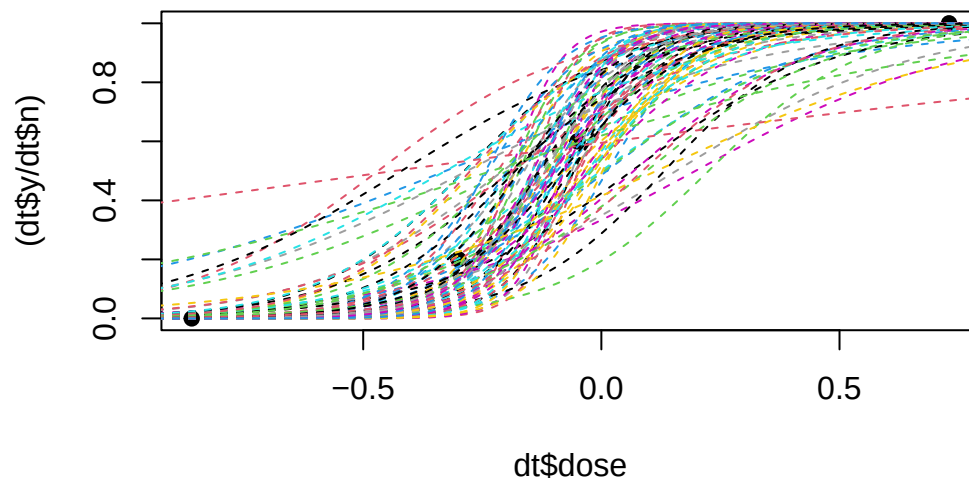
```
suppressPackageStartupMessages(library(arm))
theta_post <- invlogit(alpha + beta * xx)
y <- sapply(theta_post, \(theta) rbinom(n = 1, size = 1, prob = theta))

n_sim <- 100
xx <- seq(from = -1.0, to = 1.0, length = 200)
curves <- matrix(NA, nrow = n_sim, ncol = length(xx))

for (i in 1:n_sim) {
  num <- exp(sim_stan$alpha[i] + sim_stan$beta[i]*xx)
  curves[i, ] <- num / (1 + num)
}

plot(dt$dose, (dt$y / dt$n), pch = 19)

for (i in 1:n_sim) {
  lines(xx, curves[i, ], lty = 2, col = i)
}
```

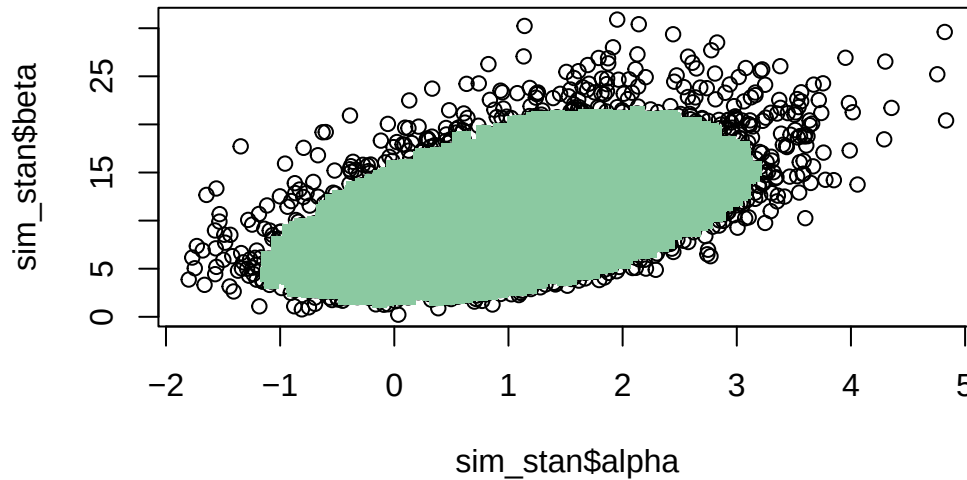


b)

```
plot(sim_stan$alpha, sim_stan$beta)

## 95% highest posterior density region
indx_hpd <- which(sim_stan$lp__ > quantile(sim_stan$lp__, 0.05))

points(sim_stan$alpha[indx_hpd], sim_stan$beta[indx_hpd], col = "#8BC9A1", pch = 15)
```



c)

For this, I can just find the ratio of my sampled betas that are greater than 0:

```
mean(beta > 0)
```

```
[1] 1
```

Because this Posterior Probability is 1, that means that the ratio of $\frac{P(H_1|D)}{P(H_2|D)}$ is infinite (divide by 0), so the Bayes Factor is Infinite.

4 BDA3 3.13

Derive equations (3.12) by completing the square:

$$p(\mu|y, \Sigma) \propto \exp\left(-\frac{1}{2}\left((\mu - \mu_0)^\top \Lambda_0^{-1}(\mu - \mu_0) + \sum_{i=1}^n (y_i - \mu)^\top \Sigma^{-1}(y_i - \mu)\right)\right)$$

We will need the following:

For a symmetric matrix Σ : $x^\top \Sigma x_0 = x_0^\top \Sigma x$

Completing the matrix square: with $x^\top (A + B)x - 2(a^\top A + b^\top B)x + (a^\top Aa + b^\top Bb)$, let $M = A + B$, $m = (M^{-1})^\top (B^\top b + A^\top a)$, $C = a^\top Aa + b^\top Bb - m^\top Mm$. Then the quadratic reduces to: $(x - m)^\top M(x - m) + C$

Also going to use the fact that the transpose of symmetric matrices (the covariance matrices) yields the same matrix.

$$\begin{aligned}
\text{exponent} &= (\mu - \mu_0)^\top \Lambda_0^{-1} (\mu - \mu_0) + \sum_{i=1}^n (y_i - \mu)^\top \Sigma^{-1} (y_i - \mu) \\
&= \mu^\top \Lambda_0^{-1} \mu - \mu^\top \Lambda_0^{-1} \mu_0 - \mu_0^\top \Lambda_0^{-1} \mu + \mu_0^\top \Lambda_0^{-1} \mu_0 + \sum_{i=1}^n [y_i^\top \Sigma^{-1} y_i - y_i^\top \Sigma^{-1} \mu - \mu^\top \Sigma^{-1} y_i + \mu^\top \Sigma^{-1} \mu] \\
&= \mu^\top \Lambda_0^{-1} \mu - 2\mu_0^\top \Lambda_0^{-1} \mu + \mu_0^\top \Lambda_0^{-1} \mu_0 + n\bar{y}^\top \Sigma^{-1} \bar{y} - 2n\bar{y}^\top \Sigma^{-1} \mu + \mu^\top \Sigma^{-1} \mu \\
&= \mu^\top (\Lambda_0^{-1} + n\Sigma^{-1}) \mu - 2(\mu_0^\top \Lambda_0^{-1} + n\bar{y}^\top \Sigma^{-1}) \mu + \mu_0^\top \Lambda_0^{-1} \mu_0 + n\bar{y}^\top \Sigma^{-1} \bar{y} \\
&\text{Using the above property of the completing the square...} \\
&\propto [\mu - ((\Lambda_0^{-1} + n\Sigma^{-1})^{-1})(n(\Sigma^{-1})\bar{y} + (\Lambda_0^{-1})\mu_0)]^\top (\Lambda_0^{-1} + n\Sigma^{-1}) [\mu - ((\Lambda_0^{-1} + n\Sigma^{-1})^{-1})(n(\Sigma^{-1})\mu_0 + (\Lambda_0^{-1}
\end{aligned}$$

This gives the sme parameterization of the book of:

$$p(\mu|y, \Sigma) \propto \exp(-\frac{1}{2}(\mu - \mu_n)^\top \Lambda_n^{-1} (\mu - \mu_n))$$

$$\text{Where: } \mu_n = (\Lambda_0^{-1} + n\Sigma^{-1})^{-1} (\Lambda_0^{-1} \mu_0 + n\Sigma^{-1} \bar{y})$$

$$\Lambda_n^{-1} = \Lambda_0^{-1} + n\Sigma^{-1}$$

5

$$P(y_i = 0|\pi, \lambda) = \pi + (1 - \pi)e^{-\lambda}$$

$$P(y_i = k|\pi, \lambda) = (1 - \pi)e^{-\lambda} \frac{\lambda^k}{k!}$$

$$\pi \in (0, 1), \lambda > 0$$

a)

To show PMF, just need to integrate/sum over support:

For the $y_i = 0$ case:

$$\pi = 0 \implies \int_0^\infty e^{-\lambda} d\lambda = -e^{-\lambda}|_0^\infty = 0 - (-1) = 1$$

$$\pi = 1 \implies 1$$

For the $y_i > 0$ case:

$$\pi = 0 \implies e^{-\lambda} \frac{\lambda^k}{k!} \text{ which is just the Poisson kernel which we know integrates correctly.}$$

$$\pi = 1 \implies 0$$

$$\text{Summing each part together: } (y_i = 0)(1) + (y_i = 0)(1) + 1 + (y_i > 0)(0) = 1$$

Thus this is a valid PMF.

b)

Let r_0 be the ratio of 0s in the dataset. We'll have the first piece (the Zero-inflated) for all the 0s we have in the dataset, represented by nr_0 , and the number of successes (non-zeros) for the second part, with the sum of the ks on the last part. This gives us:

$$l(\pi, \lambda) = nr_0 \ln(\pi + (1 - \pi)e^{-\lambda}) + n(1 - r_0)(\ln(1 - \pi) - \lambda) + n\bar{x} \ln(\lambda)$$

$$\frac{\partial}{\partial \pi} l(\pi, \lambda) = \frac{nr_0(1 - e^{-\lambda})}{\pi + (1 - \pi)e^{-\lambda}} - \frac{n(1 - r_0)}{1 - \pi}$$

```
y <- c(rep(0, 25), rep(1, 6), rep(2, 4), rep(3, 3), 5)

neg_log_lik <- function(theta, x) {
  # Harken back to 624 on setting limits for the parameters haha
  p <- 1/(1+exp(-theta[1])) # Ensures pi is between 0 and 1
  lambda <- exp(theta[2]) # Ensures lambda stays positive

  # Log-likelihood components
  loglik_zero <- log(p + (1 - p) * exp(-lambda))
  loglik_nonzero <- log(1 - p) - lambda + x * log(lambda) - lgamma(x + 1)

  # Combine
  -sum(ifelse(x == 0, loglik_zero, loglik_nonzero))
}

# I just picked some random starting values. Could probably refine with estimates based on data
mle <- optim(par = c(p = .5, lambda = 1), fn = neg_log_lik, x = y)

mle_p <- round(1/(1+exp(-mle$par[1])), 4)
mle_lambda <- round(exp(mle$par[2]), 4)
```

This gives the following numerical MLEs:

$$\hat{\pi} = 0.5495$$

$$\hat{\lambda} = 1.5934$$

c)

$$P(y_i = 0) \approx \hat{\pi} + (1 - \hat{\pi})e^{-\hat{\lambda}} = 0.5495 + (1 - 0.5495)e^{-1.5934}$$

```
p_0 <- round(mle_p + (1 - mle_p)*exp(-mle_lambda), 4)
```

This gives a probability of getting 0 fish of: 0.6411

d)

$$P(y = 1|y \geq 1) = P(y = 1)/P(y \geq 1) = P(y = 1)/(1 - P(y = 0))$$

```
p_y_eq_1 <- (1-mle_p)*exp(-mle_lambda)*mle_lambda^1 / factorial(1)
p_d <- round(p_y_eq_1/(1-p_0), 4)
```

$$P(y = 1|y \geq 1) = 0.4065$$

e)

$$p(\pi, \lambda) = L(\pi, \theta|y) \text{Beta}_{\pi}(1, 1) \text{Gamma}_{\lambda}(2, 1) \propto \prod [\pi + (1 - \pi)e^{-\lambda}I(y = 0) + (1 - \pi)e^{-\lambda}\lambda^y/y!I(y \geq 1)] \lambda e^{-\lambda}$$

```
p_seq <- seq(.01, .99, length = 100)
l_seq <- seq(.01, 10, length = 1000)
grid <- expand.grid(p_seq, l_seq)

log_lik <- function(p, lambda, x) {
  # Log-likelihood components
  loglik_zero <- log(p + (1 - p) * exp(-lambda))
  loglik_nonzero <- log(1 - p) - lambda + x * log(lambda) - lgamma(x + 1)

  # Combine
  sum(ifelse(x == 0, loglik_zero, loglik_nonzero))
}

logp <- function(p, l, y) {
  log_lik(p, l, y) + dbeta(p, 1, 1, log = TRUE) + dgamma(l, 2, 1, log = TRUE)
}

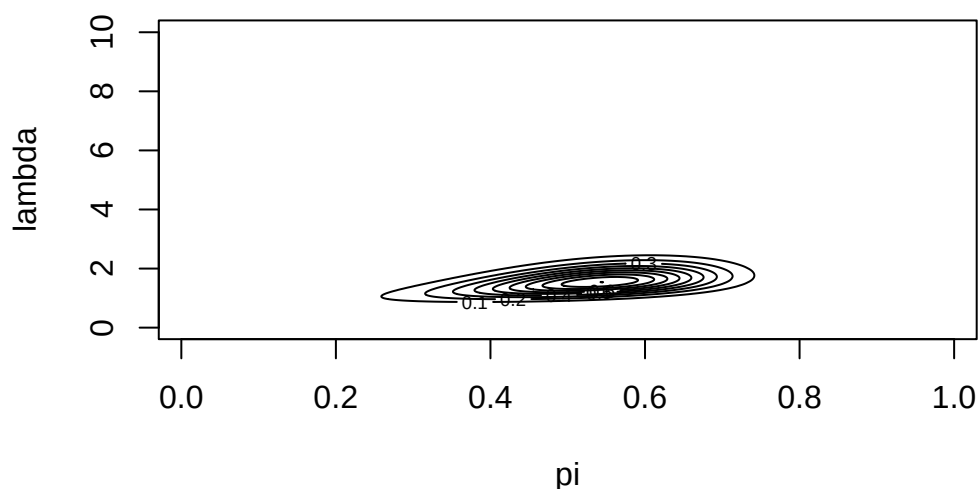
grid$logp <- mapply(logp, grid[, 1], grid[, 2], MoreArgs = list(y = y))

grid$d <- exp(grid$logp - max(grid$logp))

d <- matrix(grid$d, nrow = 100, ncol = 1000, byrow = FALSE)

contour(p_seq, l_seq, d, xlab = 'pi', ylab = 'lambda', main = 'Contour of Pi and Lambda Poster
```

Contour of Pi and Lambda Posterior



f)

Gibbs

```
logfc_pi <- function(p, lambda, x) {
  log_lik(p, lambda, x) + dbeta(p, 1, 1, log = TRUE)
}

logfc_lambda <- function(p, lambda, x) {
  log_lik(p, lambda, x) + dgamma(lambda, 2, 1, log = TRUE)
}

# Initialize things
n_pi <- 200
pi_seq <- seq(0, 1, length = n_pi)
n_lambda <- 500
lambda_seq <- seq(0, 30, length = n_lambda)

state <- list(pi = .5, lambda = 1.5)
n_iter <- 1000
sims <- as.data.frame(matrix(NA, nrow = n_iter, ncol = length(state)))
colnames(sims) <- names(state)

for (i in 1:n_iter) {
  ## Update Pi
  lp_pi <- sapply(pi_seq, \(p) logfc_pi(p = p, lambda = state$lambda, x = y))
  state$pi <- sample(pi_seq, 1, prob = exp(lp_pi - max(lp_pi))) # It auto-standardizes the prob

  ## Update Lambda
```

```

lp_lambda <- sapply(lambda_seq, \(l) logfc_lambda(p = state$pi, lambda = l, x = y))
state$lambda <- sample(lambda_seq, 1, prob = exp(lp_lambda - max(lp_lambda))) # It auto-stand

sims[i, 'pi'] <- state$pi
sims[i, 'lambda'] <- state$lambda
}

```

g)

```

pi_95_lower <- round(quantile(sims$pi, .025), 4)
pi_95_upper <- round(quantile(sims$pi, .975), 4)
lambda_95_lower <- round(quantile(sims$lambda, .025), 4)
lambda_95_upper <- round(quantile(sims$lambda, .975), 4)

```

The 95% interval for π is: 0.2863 to 0.7085 The 95% interval for λ is: 0.9018 to 2.4048

h)

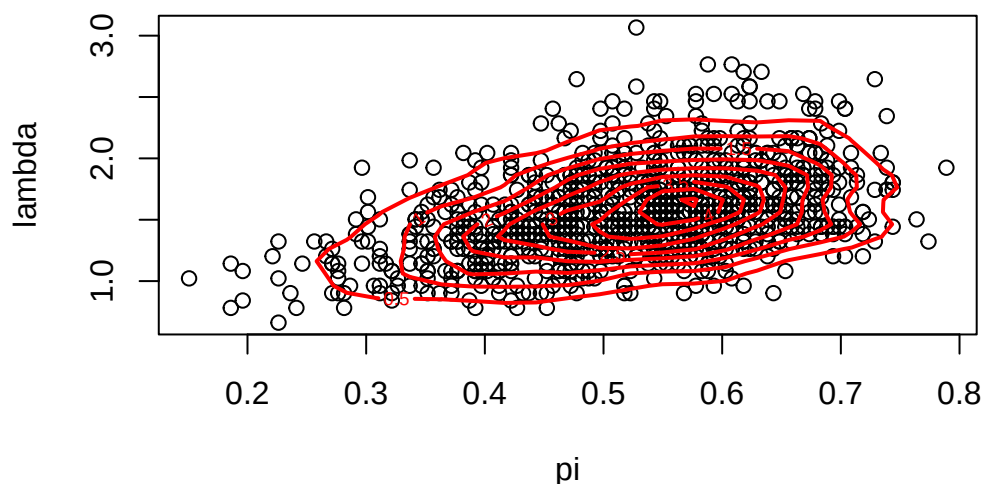
```

# Use the HPD
dens <- kde2d(x = sims[, 1], y = sims[, 2])
dens_95 <- dens[which(dens$z >= .05)]

plot(sims[, 'pi'], sims[, 'lambda'], xlab = "pi", ylab = "lambda", main = '95% Interval Contour')
contour(dens$x, dens$y, dens$z, col = "red", lwd = 2, add = TRUE)

```

95% Interval Contour Plotted on Sampled Posterior Data



The 95% interval region is the red contours. This is the region where we are 95% confident that the true π and λ values lie in.

i)

```
p <- sims[, 1]
l <- sims[, 2]
post_pred <- p + (1-p)*exp(-l)
p_y_0 <- round(mean(post_pred), 4)
p_y_0_lower <- round(quantile(post_pred, .025), 4)
p_y_0_upper <- round(quantile(post_pred, .975), 4)
```

$P(y = 0) = 0.6341$ 95% confidence interval is: 0.4817 to 0.7704

j)

```
p <- sims[, 1]
l <- sims[, 2]
post_pred <- (1-p)*exp(-l)*l/factorial(1)/(1-p_y_0)
p_y_1 <- round(mean(post_pred), 4)
p_y_1_lower <- round(quantile(post_pred, .025), 4)
p_y_1_upper <- round(quantile(post_pred, .975), 4)
```

$P(y = 1|y \geq 1) = 0.4207$ 95% confidence interval is: 0.2081 to 0.7026

k)

```
#### STAN MODEL
data_stan <- list(N = length(y), y = y)

params <- c("theta", "lambda")

n_cores <- parallel::detectCores() # Count how many cores are available
options(mc.cores = n_cores)

fit_stan <- stan(model_code = readLines("q5.stan"),
                 data = data_stan, pars = params,
                 iter = 5000, warmup = 500, thin = 5, chains = n_cores)
```

Warning in readLines("q5.stan"): incomplete final line found on 'q5.stan'

```
sim_stan <- extract(fit_stan) # collect samples from all chains

stan_p <- sim_stan$theta
stan_lambda <- sim_stan$lambda
```

STAN code:

```

data {
  int<lower=0> N;
  array[N] int<lower=0> y;
}
parameters {
  real<lower=0, upper=1> theta;
  real<lower=0> lambda;
}
model {
  for (n in 1:N) {
    if (y[n] == 0)
      target += log_sum_exp(bernoulli_lpmf(1 | theta),
                           bernoulli_lpmf(0 | theta)
                           + poisson_lpmf(y[n] | lambda));
    else
      target += bernoulli_lpmf(0 | theta)
               + poisson_lpmf(y[n] | lambda);
  }
}

```

k.g)

```

stan_pi_95_lower <- round(quantile(stan_p, .025), 4)
stan_pi_95_upper <- round(quantile(stan_p, .975), 4)
stan_lambda_95_lower <- round(quantile(stan_lambda, .025), 4)
stan_lambda_95_upper <- round(quantile(stan_lambda, .975), 4)

```

The 95% interval for π is: 0.2896 to 0.7183 The 95% interval for λ is: 0.9298 to 2.489

k.h)

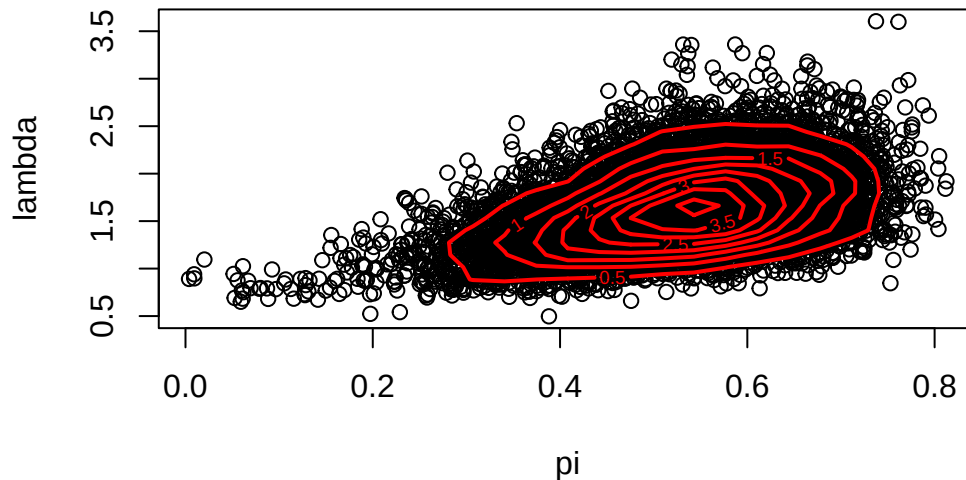
```

# Use the HPD
dens <- kde2d(x = stan_p, y = stan_lambda)
dens_95 <- dens[which(dens$z >= .05)]

plot(stan_p, stan_lambda, xlab = "pi", ylab = "lambda", main = '95% Interval Contour Plotted on
contour(dens$x, dens$y, dens$z, col = "red", lwd = 2, add = TRUE)

```

95% Interval Contour Plotted on Sampled Posterior Data



The 95% interval region is the red contours. This is the region where we are 95% confident that the true π and λ values lie in.

k.i)

```
post_pred <- stan_p + (1-stan_p)*exp(-stan_lambda)
stan_p_y_0 <- round(mean(post_pred), 4)
stan_p_y_0_lower <- round(quantile(post_pred, .025), 4)
stan_p_y_0_upper <- round(quantile(post_pred, .975), 4)
```

$P(y = 0) = 0.632$ 95% confidence interval is: 0.4784 to 0.7721

k.j)

```
post_pred <- (1-stan_p)*exp(-stan_lambda)*stan_lambda/factorial(1)/(1-stan_p_y_0)
stan_p_y_1 <- round(mean(post_pred), 4)
stan_p_y_1_lower <- round(quantile(post_pred, .025), 4)
stan_p_y_1_upper <- round(quantile(post_pred, .975), 4)
```

$P(y = 1|y \geq 1) = 0.4062$ 95% confidence interval is: 0.1935 to 0.69

Acknowledgements

Estimated time

I estimate this assignment took me in total around 14 hours to complete

Resources Used

Discussed with classmates on several problems to understand how to setup the problem or proceed with a tricky portion. Used Wikipedia for distribution PDFs. I used Stackexchange to find the general structure of a matrix 'complete the square', as well as the property allowing the switching of transposes around a symmetric matrix. I used stackexchange to see the beginning of how to find the MLEs of the Zero-inflated Poisson. I also used online resources to find how to program the STAN code for the Zero-Inflated Poisson.